

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине «Объектно-ориентированное программирование»
Тема: Создание игры в стиле ООП

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2025

ВВЕДЕНИЕ

Цель: разработка модульной и расширяемой архитектуры пошаговой тактической игры с применением современных принципов объектно-ориентированного программирования на C++, обеспечивающей гибкость модификации игровой логики и эффективное управление ресурсами.

Задачи:

1. Создать интерфейс карточки заклинания *ISpell*;
2. Реализовать интерфейс заклинания прямого урона *DirectDamageSpell*;
3. Реализовать интерфейс заклинания урона по площади *AreaDamageSpell*;
4. Реализовать интерфейс заклинания ловушки *TrapSpell*;
5. Создать класс «руки» игрока *Hand*;
6. Создать класс вражеской башни *EnemyTower*;

ОПИСАНИЕ СТРУКТУРЫ КОДА

Интерфейс *ISpell* определяет общий вид различных заклинаний в игре.

Методы интерфейса:

1. *virtual spellType name() = 0* — получение названия заклинания.
2. *virtual bool use(Field& field, Player& player, Enemy& enemy, std::pair<int, int> target) = 0* — метод использования заклинания.

Принимаемые аргументы:

Field& field — ссылка на объект игрового поля;

Player& player — ссылка на объект игрока;

Enemy& enemy — ссылка на объект врага;

std::pair<int, int> target — координаты цели заклинания.

Возвращаемое значение:

true, если заклинание было применено, иначе *false*.

3. *virtual bool use(Player& target, std::pair<int, int> tower) = 0* — метод использования заклинания для вражеской башни.

Принимаемые аргументы:

Player& target — ссылка на объект игрока;

std::pair<int, int> tower — координаты вражеской башни.

Возвращаемое значение:

true, если заклинание было применено, иначе *false*.

Класс *DirectDamageSpell* реализует интерфейс заклинания прямого урона, наносящее урон врагу в достижимом радиусе.

Поля класса:

- *int range* — радиус действия заклинания;
- *int damage* — наносимый урон;

Методы класса:

1. *DirectDamageSpell()* — конструктор по умолчанию. Инициализирует значения *range = 3*, *damage = 2*.
2. *spellType name()* - метод, возвращающий название заклинания.
3. *bool use(Field& field, Player& player, Enemy& enemy, std::pair<int, int> target)* — метод использования заклинания. Наносит урон врагу в указанной клетке, если она находится в пределах допустимого диапазона.

Принимаемые аргументы:

Field& field — ссылка на объект игрового поля;

Player& player — ссылка на объект игрока;

Enemy& enemy — ссылка на объект врага;

std::pair<int, int> target — координаты цели заклинания.

Возвращаемое значение:

true, если заклинание было применено, иначе *false*.

4. *bool use(Player& target, std::pair<int, int> tower)* — метод использования ослабленной версии заклинания для вражеской башни (игрок получает половину заданного в конструкторе урона). При нахождении игрока в определенной области башни раз в 3 хода он получает урон.

Принимаемые аргументы:

Player& target — ссылка на объект игрока;

std::pair<int, int> tower — координаты вражеской башни.

Возвращаемое значение:

true, если заклинание было применено, иначе *false*.

Класс *AreaDamageSpell* реализует интерфейс заклинания урона по площади, наносящее урон по области 2 на 2 клетки в допустимом радиусе.

Поля класса:

- *int range* — радиус действия заклинания;

- *int damage* — наносимый урон;

Методы класса:

1. *AreaDamageSpell()* — конструктор по умолчанию. Инициализирует значения *range = 4*, *damage = 1*.
2. *spellType name()* - метод, возвращающий название заклинания.
3. *bool use(Field& field, Player& player, Enemy& enemy, std::pair<int, int> target)* — метод использования заклинания. Наносит урон в области 2x2 клетки, если область находится в допустимом диапазоне.

Принимаемые аргументы:

Field& field — ссылка на объект игрового поля;

Player& player — ссылка на объект игрока;

Enemy& enemy — ссылка на объект врага;

std::pair<int, int> target — координаты верхней левой клетки области применения заклинания.

Возвращаемое значение:

true, если была попытка применения заклинания в указанном диапазоне, иначе *false*.

4. *bool use(Player& target, std::pair<int, int> tower)* — переопределение метода для вражеской башни. Поскольку башня не может использовать этот метод, всегда возвращается значение *false*.

Класс *TrapSpell* реализует интерфейс заклинания ловушки. При наступлении врага на клетку с ловушкой он получает урон, а ловушка исчезает.

Поля класса:

- *int damage* — наносимый урон;

Методы класса:

1. *TrapSpell()* — конструктор по умолчанию. Инициализирует значение *damage = 1*.
2. *spellType name()* - метод, возвращающий название заклинания.
3. *bool use(Field& field, Player& player, Enemy& enemy, std::pair<int, int> target)* — метод использования заклинания. Устанавливает ловушку на указанной клетке, если она проходима.

Принимаемые аргументы:

Field& field — ссылка на объект игрового поля;

Player& player — ссылка на объект игрока;

Enemy& enemy — ссылка на объект врага;

std::pair<int, int> target — клетка для размещения ловушки.

Возвращаемое значение:

true, если ловушка размещена, иначе *false*.

4. *bool use(Player& target, std::pair<int, int> tower)* — переопределение метода для вражеской башни. Поскольку башня не может использовать этот метод, всегда возвращается значение *false*.

Класс *EnemyTower* осуществляет применение ослабленной версии заклинания прямого урона относительно игрока, находящегося в пределах допустимой области вражеской башни.

Поля класса:

- *std::pair<int, int> towerPosition* — координаты вражеской башни.

Методы класса:

1. *EnemyTower()* - конструктор по умолчанию. Устанавливает начальные координаты башни в точку (-1; -1).
2. *bool attackTarget(Player& target)* — метод атаки игрока ослабленной версией прямого урона.

Принимаемый аргумент:

Player& target — ссылка на игрока.

Возвращаемое значение:

true, если заклинание было применено, иначе *false*.

3. *void setCoordinates(int x, int y)* — установка новых координат башни.

Принимаемые аргументы:

int x, int y — новые координаты.

4. *std::pair<int, int> getCoordinates() const* — получение текущих координат башни.

Возвращаемое значение:

std::pair<int, int> coords — координаты башни.

Класс *Hand* управляет картами заклинаний игрока.

Поля класса:

- *int size* — максимальное количество заклинаний, которые может хранить рука.
- *int fullness* — заполненность руки.
- *std::vector<ISpell*> spellCards* — имеющиеся карты заклинаний.

Методы класса:

1. *Hand(int size)* — конструктор по умолчанию. Инициализирует заданное значение размера руки, устанавливает *fullness = 0*, добавляет первое заклинание.

2. *int getFullness() const* — метод получения текущей заполненности руки.

Возвращаемое значение:

int current_fullness — заполненность руки.

3. *void setFullness(int value)* — установка нового значения заполненности руки.

Принимаемый аргумент:

int value — заполненность руки.

4. *int getSize() const* — получение размера руки.

Возвращаемое значение:

int current_size — текущий размер руки.

5. *void setSize(int value)* — установка размера руки.

Принимаемый аргумент:

int value — новый размер руки.

6. *SpellType getNameSpellCard(int coord) const* — получение название заклинания.

Принимаемый аргумент:

int coord — номер заклинания в руке.

Возвращаемое значение:

spellType name — название заклинания.

7. *bool hasSpell(int index)* — проверка наличия заклинания по номеру в руке.

Принимаемый аргумент:

int index — номер заклинания в руке.

Возвращаемое значение:

true, если заклинание есть, иначе *false*.

8. *bool addSpellCard()* - метод добавления в руку карты заклинания случайного типа.

Возвращаемое значение:

true, если карта добавлена, иначе *false*.

9. *bool cast(int spellIndex, Field& field, Player& player, Enemy& enemy, std::pair<int, int> target)* — метод вызова заклинания.

Принимаемые аргументы:

int spellIndex — индекс заклинания в руке;

Field& field — ссылка на объект игрового поля;

Player& player — ссылка на объект игрока;

Enemy& enemy — ссылка на объект врага;

std::pair<int, int> target — координаты цели.

Возвращаемое значение:

true, если заклинание применено, иначе *false*.

10. *~Hand()* - деструктор. Очищает память от всех заклинаний в руке.

ОБОСНОВАНИЕ АРХИТЕКТУРНЫХ РЕШЕНИЙ

ОБОСНОВАНИЕ АРХИТЕКТУРНЫХ РЕШЕНИЙ

Интерфейс *Ispell* является шаблоном для всех заклинаний, что упрощает добавление новых типов магических карточек через единый набор методов. Каждое заклинание имеет собственные параметры и выполняет определенную задачу.

Класс *Hand* реализует логику работы с заклинаниями, позволяет генерировать случайные типы карт для добавления в руку игрока, инкапсулирует управление указателями.

Класс *GameManager* является центральным узлом и координирует взаимодействие всех компонентов игры. Добавлены методы покупки и применения заклинания через объект игрока. За удары врагов игрок получает монеты, которые можно потратить на покупку магических карт.

Взаимосвязь классов представлена на UML-диаграмме (см. Приложение).

ТЕСТИРОВАНИЕ

```
#include <gtest/gtest.h>
#include "../include/hand.hpp"
#include "../include/directDamageSpell.hpp"
#include "../include/areaDamageSpell.hpp"
#include "../include/trapSpell.hpp"
#include "../include/field.hpp"
#include "../include/player.hpp"
#include "../include/enemy.hpp"

TEST(DirectDamageSpellTest, SuccessfulAttackInRange) {
    DirectDamageSpell spell;
    Field field(10, 10);
    Player player;
    Enemy enemy;

    player.setCoordinates(2, 2);
    enemy.setCoordinates(4, 2);
    enemy.setHealth(5);
    field.setCellCharacter(4, 2, character::ENEMY);

    bool result = spell.use(field, player, enemy, {4, 2});
    EXPECT_TRUE(result);
    EXPECT_EQ(enemy.getHealth(), 3);
}

TEST(DirectDamageSpellTest, AttackOutOfRange) {
```

```

Enemy enemy;

player.setCoordinates(0, 0);
enemy.setCoordinates(4, 4);
enemy.setHealth(5);
field.setCellCharacter(4, 4, character::ENEMY);

bool result = spell.use(field, player, enemy, {4, 4});
EXPECT_FALSE(result);
EXPECT_EQ(enemy.getHealth(), 5);
}

TEST(DirectDamageSpellTest, AttackEmptyCell) {
    DirectDamageSpell spell;
    Field field(10, 10);
    Player player;
    Enemy enemy;

    player.setCoordinates(2, 2);
    enemy.setHealth(5);

    bool result = spell.use(field, player, enemy, {3, 2});
    EXPECT_FALSE(result);
    EXPECT_EQ(enemy.getHealth(), 5);
}

TEST(DirectDamageSpellTest, TowerAttackInRange) {
    DirectDamageSpell spell;
    Player player;

    player.setCoordinates(2, 2);
    player.setHealth(10);

```

```

        std::pair<int, int> tower = {3, 2};

        bool result = spell.use(player, tower);
        EXPECT_TRUE(result);
        EXPECT_EQ(player.getHealth(), 9);
    }

TEST(AreaDamageSpellTest, SuccessfulAreaAttack) {
    AreaDamageSpell spell;
    Field field(10, 10);
    Player player;
    Enemy enemy;

    player.setCoordinates(0, 0);
    enemy.setCoordinates(2, 2);
    enemy.setHealth(3);

    field.setCellCharacter(2, 3, character::ENEMY);

    bool result = spell.use(field, player, enemy, {2, 2});
    EXPECT_TRUE(result);
    EXPECT_EQ(enemy.getHealth(), 2);
}

TEST(AreaDamageSpellTest, AreaAttackOutOfRange) {
    AreaDamageSpell spell;
    Field field(10, 10);
    Player player;
    Enemy enemy;

    player.setCoordinates(0, 0);
    enemy.setCoordinates(6, 6);

```

```

    enemy.setHealth(3);
    field.setCellCharacter(6, 6, character::ENEMY);

    bool result = spell.use(field, player, enemy, {6, 6});
    EXPECT_FALSE(result);
    EXPECT_EQ(enemy.getHealth(), 3);
}

TEST(AreaDamageSpellTest, TowerAttackNotImplemented) {
    AreaDamageSpell spell;
    Player player;

    player.setHealth(10);
    std::pair<int, int> tower = {0, 0};

    bool result = spell.use(player, tower);
    EXPECT_FALSE(result);
    EXPECT_EQ(player.getHealth(), 10);
}

TEST(TrapSpellTest, SuccessfulTrapPlacement) {
    TrapSpell spell;
    Field field(10, 10);
    Player player;
    Enemy enemy;

    player.setCoordinates(2, 2);

    bool result = spell.use(field, player, enemy, {3, 3});
    EXPECT_TRUE(result);
    EXPECT_EQ(field.getCellCharacter(3, 3), character::TRAP);
}

```

```

TEST(TrapSpellTest, TrapPlacementOnOccupiedCell) {
    TrapSpell spell;
    Field field(10, 10);
    Player player;
    Enemy enemy;

    player.setCoordinates(2, 2);
    field.setCellCharacter(3, 3, character::ENEMY);

    bool result = spell.use(field, player, enemy, {3, 3});
    EXPECT_FALSE(result);
    EXPECT_EQ(field.getCellCharacter(3, 3), character::ENEMY);
}

```

```

TEST(TrapSpellTest, TrapPlacementOnObstacle) {
    TrapSpell spell;
    Field field(10, 10);
    Player player;
    Enemy enemy;

    player.setCoordinates(2, 2);

    bool result = spell.use(field, player, enemy, {0, 0});
    EXPECT_FALSE(result);
}

```

```

TEST(TrapSpellTest, TowerAttackNotImplementedForTrap) {
    TrapSpell spell;
    Player player;

    player.setHealth(10);
}

```

```

    std::pair<int, int> tower = {0, 0};

    bool result = spell.use(player, tower);
    EXPECT_FALSE(result);
    EXPECT_EQ(player.getHealth(), 10);
}

```

```

TEST(HandTest, AddSpellCard) {
    Hand hand(2);

    EXPECT_EQ(hand.getFullness(), 1);

    bool result = hand.addSpellCard();
    EXPECT_TRUE(result);
    EXPECT_EQ(hand.getFullness(), 2);
    EXPECT_TRUE(hand.hasSpell(0));
    EXPECT_TRUE(hand.hasSpell(1));

    result = hand.addSpellCard();
    EXPECT_FALSE(result);
    EXPECT_EQ(hand.getFullness(), 2);
}

```

```

int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}

```

Запуск тестов:

```

[=====] 12 tests from 4 test suites ran. (0 ms total)
[ PASSED ] 12 tests.

```

Все проверки пройдены, программа работает корректно.

ЗАКЛЮЧЕНИЕ

В результате лабораторной работы была создана система взаимосвязанных классов, каждый из которых выполняет строго определенную функцию и может быть модифицирован независимо от других компонентов системы; создан интерфейс заклинания, реализованы различные типы магических карточек, выполняющих конкретную задачу. Добавлен класс руки игрока, позволяющий хранить одновременно несколько карточек и оперировать ими в процессе игры. Реализована логика получения монет за удары врага и покупка заклинаний. Создан класс вражеской башни, раз в 3 хода наносящий урон игроку, находящемуся в допустимом радиусе. Соблюдены принципы инкапсуляции, исключено дублирование данных, предотвращены небезопасные операции с указателями. Программа протестирована и работает корректно.

ПРИЛОЖЕНИЕ

